# Nasazení a provoz aplikací

## Deployment, provoz a monitoring

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

17.08.2026

CC BY-NC-SA 4.0

# Úvod

# Od localhost k produkci

## Co se stane s vaší Express aplikací, když zavřete notebook?

Zmizí

`localhost:3000` vidíte jen vy – a jen dokud počítač běží

**Produkce** znamená: aplikace běží **pořád**, **pro všechny** – i když spíte

# Od localhost k produkci

Minule naše pipeline postavila otestovaný Docker image

Skončili jsme otázkou: **kam se nasazuje a co je v `deploy.sh`?**

Dnes příběh dokončíme:

- **Nasazení** – kde a jak aplikace poběží
- **Provoz** – logy, aktualizace, pády
- **Monitoring** – jak se o problému dozvědět dřív než uživatelé

# Od localhost k produkci

Kolik podle vás stojí firmu **jedna minuta** výpadku?

V průměru přibližně **5 600 USD** – za minutu

Proto se provoz aplikací (**operations**) bere stejně vážně jako vývoj

*Pozn.: Zdroj: Gartner (2014), průměr napříč odvětvími – řádový odhad*

# Kde aplikace poběží

# Server

## Co si představíte pod pojmem server?

Obyčejný počítač, který běží **nepřetržitě** a je **dostupný z internetu**

- Žádná magie – stejný Linux, jaký jsme viděli v kontejnerech
- Liší se jen tím, že má **veřejnou IP adresu**
- A že ho (většinou) nikdy neuvidíte – stojí v datacentru

# Server

**VPS** (*Virtual Private Server*) je pronajatý virtuální stroj v datacentru

- Dostanete IP adresu a root přístup
- Platíte měsíčně, od ~150 Kč (Hetzner, DigitalOcean, …)
- O hardware, elektřinu a konektivitu se stará provozovatel

Pro vlastní projekty doporučuji začít právě s VPS – naučíte se nejvíc

# Možnosti nasazení

VPS není jediná cesta:

| | Spravujete | Cena | Příklad |
|---|---|---|---|
| Vlastní hardware | všechno | vysoká vstupní | serverovna firmy |
| **VPS** | OS a výš | ~150 Kč/měs. | Hetzner |
| PaaS | jen aplikaci | dražší za výkon | Railway, Render |
| Serverless | jen funkce | platba za volání | AWS Lambda |

Čím míň spravujete, tím víc platíte za pohodlí

*Pozn.: Serverless architekturu probere kolega v příštím pololetí*

# Připojení k serveru

K serveru se připojíme přes **SSH**:

```
$ ssh root@203.0.113.42
Welcome to Ubuntu 24.04 LTS
root@server:~$
```

SSH klíč už máte! – vygenerovali jste si ho v první hodině pro GitLab

Stejný pár klíčů poslouží i tady – veřejný klíč se nahraje na server

# Připojení k serveru

## Co na server nainstalujeme?

Jen **Docker**

- Aplikace, databáze i vše ostatní poběží v kontejnerech
- Server zůstane čistý a přenositelný
- Vzpomeňte si: „funguje mi to lokálně“ ≈ funguje všude

Přesně pro tenhle moment jsme se Docker učili

# První nasazení

# Spuštění aplikace

Minule měl `compose.yml` u aplikace `build: .`

Na serveru nebudeme nic stavět – image už **postavila pipeline**:

```yaml
services:
  app:
    image: git.delta-vyuka.cz:5050/firma/app:a1b2c3d
    restart: always
    env_file: .env
```

Služba `db` s databází zůstává stejná jako minule

# Spuštění aplikace

Přihlásíme se k registry a spustíme:

```
$ docker login git.delta-vyuka.cz:5050
$ docker compose up -d
[+] Running 2/2
 ✔ Container app  Started
 ✔ Container db   Started
```

```
$ curl http://203.0.113.42:3000/health
{"status":"ok"}
```

Aplikace **běží na internetu!** Ale…

# Chybějící kousky

## Dali byste zákazníkovi adresu `http://203.0.113.42:3000`?

Co všechno chybí:

- **Doména** – lidé si pamatují jména, ne čísla
- **HTTPS** – prohlížeče označí `http://` jako „nezabezpečené“
- **Standardní porty** – web patří na 80 a 443, ne na 3000

# Chybějící kousky

Doménu na IP adresu překládá **DNS**:

| Záznam | Typ | Hodnota |
|---|---|---|
| `example.com` | A | `203.0.113.42` |
| `www.example.com` | CNAME | `example.com` |

Záznam nastavíte u registrátora domény (kde jste ji koupili)

Zbývá HTTPS a porty – na to potřebujeme nový kousek skládačky

# Reverse proxy

# Definice

Jak dostat požadavky z portu 443 do aplikace na portu 3000?

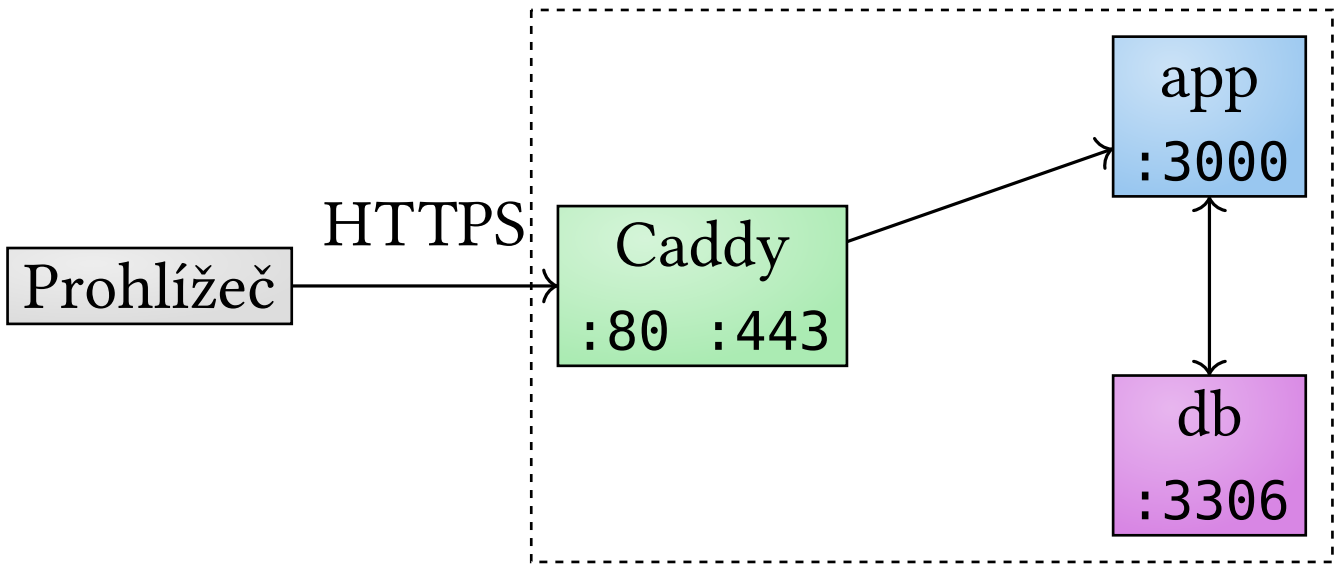
A co když na jednom serveru běží **víc aplikací**?

**Reverse proxy** je server, který stojí **před** aplikacemi – přijímá všechny požadavky a rozděluje je dál

**Jakou analogii si můžete představit?**

Recepční v budově – návštěvník řekne, za kým jde (doménu), a recepční ho zavede do správné kanceláře (kontejneru)

# Architektura




Vše v rámečku běží na **jednom VPS** jako kontejnery – zvenku je vidět jen proxy

# Nástroje

Nejznámější reverse proxy:

- **nginx** (2004) – průmyslový standard, potkáte ho všude
- **Caddy** (2015) – automatické HTTPS, jednoduchá konfigurace
- **Traefik** – konfigurace přímo z Docker labelů

Pro začátek doporučuji **Caddy** – uvidíte za chvíli proč

# Caddy

Celá konfigurace – soubor `Caddyfile`:

```
example.com {
    reverse_proxy app:3000
}
```

**Tři řádky** – a máme:

- přesměrování domény na aplikaci
- porty 80 i 443
- HTTPS certifikát – **automaticky, zdarma**

# Caddy

Odkud se ten certifikát bere?

**Let's Encrypt** (2015) – nezisková autorita vydávající certifikáty **zdarma**

- Před ní stál certifikát tisíce korun ročně
- Dnes má HTTPS přes 90 % webů
- Caddy si certifikát vyžádá i **obnoví** sám – platí 90 dní

*Pozn.: nginx umí totéž s nástrojem certbot, jen to musíte nastavit ručně*

# Kompletní sestava

Do `compose.yml` přibude třetí služba:

```yaml
services:
  caddy:
    image: caddy:2
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - caddy-data:/data
```

`app` a `db` zůstávají beze změny – jen jim `ports:` **nedáváme**

# Kompletní sestava

**Všimněte si**: porty ven vystavuje **jen Caddy**

- `app` a `db` nemají žádné `ports:`
- Kontejnery se vidí navzájem přes vnitřní síť (`app:3000`)
- Databáze **není dostupná z internetu** – a to je dobře

Na serveru tak zbývají tři soubory: `compose.yml`, `Caddyfile` a `.env`

# Konfigurace a secrets

# Environment variables

Kde vzít heslo k databázi? V kódu být nesmí:

```
// WRONG: secret committed to git
const db = mysql.createConnection({
  host: "db",
  password: "hunter2",
});
```

## Co je na tom špatně?

- Heslo vidí každý, kdo vidí repozitář
- Nejde změnit bez nového nasazení
- Dev a produkce potřebují **různé** hodnoty

# Environment variables

**Řešení**: konfigurace patří do **proměnných prostředí**

```
// Correct: configuration from environment
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  password: process.env.DB_PASSWORD,
});
```

Kód je pro všechna prostředí **stejný** – hodnoty dodá až prostředí, kde běží

*Pozn.: Tento princip popisuje manifest The Twelve-Factor App (Heroku, 2011)*

# Environment variables

Odkud se hodnoty vezmou? Na serveru je soubor `.env`:

```
DB_HOST=db
DB_PASSWORD=s3cret-generated-password
NODE_ENV=production
```

Docker Compose ho načte přes `env_file: .env`

**Nikdy** necommitujte `.env`! – patří do `.gitignore`, na serveru vzniká ručně

# Environment variables

K čemu je `NODE_ENV=production`?

Aplikace podle něj pozná, kde běží:

- Express vypne podrobné chybové stránky
- Knihovny zapnou optimalizace
- A vzpomeňte si na TypeORM: `synchronize: true` **nikdy** v produkci!

Jedna proměnná, jiné chování – **stejný kód**

# Provoz

# Logy

Aplikace běží na serveru a my u ní nesedíme. **Jak zjistit, co dělá?**

Kontejnery píší na standardní výstup a Docker ho sbírá:

```
$ docker compose logs -f app
app  | GET /users 200 12ms
app  | GET /users/42 404 3ms
app  | POST /orders 201 87ms
```

Poznáváte výstup? – náš **logging middleware** z přednášky o Expressu

Při problému jsou logy **první místo, kam se podíváte**

# Pády a restarty

## Co když aplikace spadne ve tři ráno?

```
services:
  app:
    restart: always
```

Docker spadlý kontejner **sám nastartuje** – a po restartu serveru také

Jeden řádek = spíte klidněji

**Pozor**: restart řeší následek, ne příčinu – pád si ráno dohledejte v lozích

# Aktualizace aplikace

Pipeline z minulé přednášky postavila novou verzi image. **Jak ji dostat na server?**

```
$ docker compose pull app
$ docker compose up -d app
```

Stáhne novou verzi a vymění kontejner – data v databázi zůstávají

A teď už víte, co je v `deploy.sh`: **přesně tyhle dva příkazy**, spuštěné přes SSH

*Pozn.: Výměna znamená pár sekund výpadku. Nasazení bez výpadku (zero-downtime) řeší orchestrátory jako Kubernetes – mimo rozsah předmětu*

# Monitoring

# Zjištění výpadku

## Jak se dozvíte, že váš web nefunguje?

Nejčastější odpověď v praxi: od uživatelů – nebo od šéfa

To je **pozdě**

O výpadku se nechcete dozvědět od zákazníka!

# Uptime monitoring

Aplikaci přidáme jednoduchý **healthcheck endpoint**:

```
app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});
```

Externí služba se na něj ptá **každou minutu**:

- Když neodpoví, pošle e-mail nebo SMS
- UptimeRobot, Better Stack – základ **zdarma**
- Nastavení zabere 5 minut

Nejlevnější pojistka, jakou kdy koupíte

# Sledování chyb

Výpadek není jediný problém – aplikace může běžet a přitom **házet chyby**

**Error tracking** služby zachytí každou výjimku:

- Stack trace, request, prohlížeč uživatele
- Seskupí stejné chyby a spočítá výskyty
- Doporučuji **Sentry** – pro malé projekty zdarma

Rozdíl oproti logům: chyba vás **sama najde**, nemusíte ji hledat

# Metriky serveru

Občas se hodí vědět, jak se serveru daří:

```
$ docker stats
NAME    CPU %   MEM USAGE
app     2.3%    85 MiB
db      1.1%    380 MiB

$ df -h /
/dev/sda1   38G   31G   5.2G   86%
```

**Co serveru dochází nejčastěji? Disk** – logy a staré image rostou (`docker system prune`!)

*Pozn.: V praxi metriky sbírá Prometheus a kreslí Grafana – na vlastní VPS stačí příkazy výše*

# Zálohy

## Server shořel a poskytovatel ho neobnoví. Co vám zbylo?

Jen to, co je **mimo server**:

- Kód – v gitu ✓
- Image – v registry ✓
- Databáze – **nikde** ✗

```
$ docker compose exec db mysqldump -u root \
    -p"$DB_ROOT_PASSWORD" shop > backup-2026-07-08.sql
```

Spouštět denně (cron) a ukládat **mimo server**

Záloha, kterou jste nikdy neobnovili,

není záloha!

# Nasazení v praxi

# Produkční checklist

| Otázka | Naše odpověď |
|---|---|
| Běží to i po pádu a restartu? | `restart: always` |
| Je spojení šifrované? | Caddy + Let's Encrypt |
| Jsou tajemství mimo git? | `.env` na serveru, CI/CD Variables |
| Poznáme, že to spadlo? | healthcheck + uptime monitoring |
| Vidíme chyby uživatelů? | Sentry |
| Přežijeme ztrátu serveru? | denní dump databáze mimo server |

Šest otázek – projděte si je u každého projektu, který nasazujete

# Produkční checklist

Jedno pravidlo zkušených na závěr:

## Nenasazujte v pátek odpoledne

Ne že by pipeline v pátek fungovala hůř – ale případný problém řešíte o víkendu

Nasazení má být **nudná rutina**, ne adrenalin – přesně k tomu jsme celou automatizaci stavěli

# Shrnutí

# Klíčové poznatky

Co jsme se dnes naučili:

- Produkce = **VPS s Dockerem** – stačí `compose.yml`, `Caddyfile` a `.env`
- **Reverse proxy** (Caddy) řeší domény, porty i HTTPS
- Konfigurace a tajemství žijí v **proměnných prostředí**
- Nasazení = `docker compose pull && up -d` – celý `deploy.sh`
- **Monitoring a zálohy** – o problému chcete vědět první

# Klíčové poznatky

Vyzkoušejte si sami:

- **GitHub Student Developer Pack** – kredity na DigitalOcean a Azure zdarma
- Nebo VPS za ~150 Kč měsíčně – méně než jedna pizza
- Nasaďte svůj domácí úkol: server, Docker, Caddy, doména

Vlastní aplikace běžící na internetu je **nejlepší řádek v životopisu**, jaký teď můžete mít

# Užitečné zdroje

## Dokumentace

- Caddy: https://caddyserver.com/docs/
- Docker Compose v produkci: https://docs.docker.com/compose/production/

## Doporučená četba

- The Twelve-Factor App: https://12factor.net/
- Let's Encrypt – jak funguje: https://letsencrypt.org/how-it-works/